

Testing in Python

Fare i test in Python è molto importante ed è una parte essenziale nel software development: ci assicura che il codice fa quello che devo fare e ci permette di risparmiare tempo e denaro. Di solito si ha poco tempo a disposizione per stare dietro agli errori, quindi appena si sviluppa un pezzo di codice si passa alla fase di test per assicurarci che nel caso si trovino degli errori questi possono venire fixati subito, anche perché se troppi errori si accumulano può diventare un problema. Quindi in sostanza bisogna scrivere piccoli pezzi di codice di test anche detti unità di test (test unity), quindi permettono di assicurarci che quel blocco di codice lavori come ci aspettiamo.

Per fare i test python preseta due framework: Pytest e unittest

Per fare i test dobbiamo capire come funziona assert:

comando built-in che consente di asserire/ affermare che una certa condizione è vera.

Se poi la condizione che noi asseriamo essere vera è falsa allora viene sollevato un `assertionError`.

In un file chiamato `prova.py`, scriviamo:



Python

```
1 assert 1>0
```

Se facciamo runnare il programma non succede nulla, se si modifica l'asserzione:



Python

```
1 assert 1 < 0
```

Se si runna il codice viene sollevato un `AssertionError` questo perché si sta spacciando una condizione falsa come vera e quindi la condition assertemtn falisce.

Si può customizzare i messaggi con l'assert:



Python

```
1 assert 1 < 0, 'The Condition is False'
```

Quando si runna il programma viene sollevato l'errore di assertion e viene visualizzato così:



Shell

```
1 Traceback (most recent call last):
2   File "c:\Users\Project Lead\Desktop\Esercizi Programazione
  ITS\Progetti_Py\Lezione 14 pytest\prova.py", line 1, in <module>
3     assert 1 < 0, "The Condition is False"
4         ^^^^^
5 AssertionError: The Condition is False
```

La sintassi

La sintassi dell'assert è:



Python

```
1 assert <condizione che deve essere testata>, <messaggio di errore
   che deve essere visualizzata>
```

Pytest

Cancelliamo il file `prova.py`.

Andiamo sul terminale di Vs Code per installare PyTest:



Shell

```
1 pip install -U pytest
```

Dopodiché controllare se l'installazione è avvenuta con successo:



Shell

```
1 pytest --version
2 pytest <numeroVersione>
```

Convenzioni per l'organizzazione dei test con Pytest

Quando si scrivono test automatici in Python utilizzando **Pytest**, è fondamentale seguire alcune convenzioni che facilitano:

- Il riconoscimento automatico dei test da parte del framework.

- Una struttura ordinata del progetto.
- L'evitamento di conflitti di importazione.

📁 Struttura consigliata del progetto

Si raccomanda l'uso di una struttura a cartelle coerente. In particolare:

- Creare una directory dedicata ai test, denominata `tests/`.
- All'interno della directory `tests/`, è possibile creare sottocartelle organizzate per modulo o funzionalità.
- Ogni file contenente test deve iniziare con il prefisso `test_`.

❌ Evitare il nome `test` per la directory

È **fortemente sconsigliato** denominare la directory dei test con il nome `test`, poiché:

- `test` è un modulo della libreria standard di Python.
- L'utilizzo di tale nome potrebbe generare **conflitti nei nomi dei moduli**, problemi di importazione o malfunzionamenti durante l'esecuzione dei test.

✅ **Usare sempre la forma plurale:** `tests`.

Esempio di struttura corretta

```

1  project_root/
2  |
3  ├── src/
4  |   └── frazioni.py
5  |
6  ├── tests/
7  |   ├── __init__.py
8  |   └── test_frazioni.py
9  |
10 └── requirements.txt
11
```

Convenzioni nei nomi dei file e delle funzioni

- **File di test:** devono iniziare con `test_` → es. `test_frazioni.py`.

- **Funzioni di test:** devono anch'esse iniziare con `test_` → es. `test_valore_frazione()`.

Il primo test di pytest

consideriamo la struttura dei file nella cartella lezione 14:

```
CSS
1  my_project/
2  |— weather.py
3  |— tests/
4     |— test_weather.py
```

Nel file `weather.py` troviamo scritto il seguente codice:

```
Python
1  def check_weather(temperature: float) -> str:
2      if temperature > 20:
3          return "hot"
4      elif 10 < temperature <= 20:
5          return "average"
6      else:
7          return "cold"
```

Mentre nel file `test_weather.py`:

```
Python
1  from my_project.weather import check_weather
2  # passed
3  def test_check_weather():
4      assert check_weather(21.00) == "hot", 'temperatures greater
   than 20 degree \
5      must be considered as hot'
```

Per runnare questo file bisogna andare sul terminale di vs code, andare sul nome del file, click destro sul nome del file e cliccare alla voce `Open in integrated terminal` dopodiche nel terminale scrivere:

```
Shell
>_
```

```
1 pytest my_projects/tests/test_weather.py
```

L'output sarà:

```
collected 1 item
my_project\tests\test_weather.py . [100%]
----- 1 passed in 0.03s -----
```

Output pytest.png

Analizziamo questa immagine:

collected 1 item : pytest trova 1 test per runnare il file test_weather.py .

Il punto verde dopo pytest my_projects/tests/test_weather.py significa che il test è stato passato con successo. Ogni punto rappresenta un test passato.

100% : si riferisce all'avanzamento complessivo dell'esecuzione di tutti i casi di test.

1 passed in 0.03s===== : Tutti i test (1 su 1) sono stati eseguiti con successo in 0,03 secondi.

Caso inverso se il test fallisce

Nel file test_weather.py :



Python

```
1 from my_project.weather import check_weather
2 # failed
3 def test_check_weather():
4     assert check_weather(5.00) == "average", 'temperatures between 10
   and 20 degree \
5     must be considered as average temperature'
```

Per runnare il file test_weather.py :



Shell

```
1 pytest my_projects/tests/test_weather.py
```

```
collected 1 item

my_project\tests\test_weather.py F [100%]

===== FAILURES =====
test_check_weather

def test_check_weather():
> assert check_weather(5.00) == "average", "temperatures between 10 and 20 degree must be considered as average temperature"
E   AssertionError: temperatures between 10 and 20 degree must be considered as average temperature
E   assert 'cold' == 'average'
E
E   - average
E   + cold

my_project\tests\test_weather.py:10: AssertionError
===== short test summary info =====
FAILED my_project\tests\test_weather.py::test_check_weather - AssertionError: temperatures between 10 and 20 degree must be considered as average temperature
===== 1 failed in 0.30s =====
```

Output pytest fail.png

La f rossa che segue `my_project\tests\test_weather.py` significa che il test è fallito

`test_chckc_weather` : è il nome del test che la sezione di errore mostra il nome del test che è fallito.

- - average : (-) il valore aspettato.
- + cold : (+) il valore attuale.

`\=====short test summary info\=====` :

Questo è una versione compatta del report di fallimento; dice esattamente quali test sono falliti e quali tipi di errore si è verificato (in questo caso `AssertionError`).

`1 failed in 0.30s` : la conslusione del test; pytest runna 1 test fallito e l'intera esecuzione prende 0.26 secondi.

Ora continuiamo a scrivere altri `test_check_weather` nel file `test_weather.py` :

```
Python

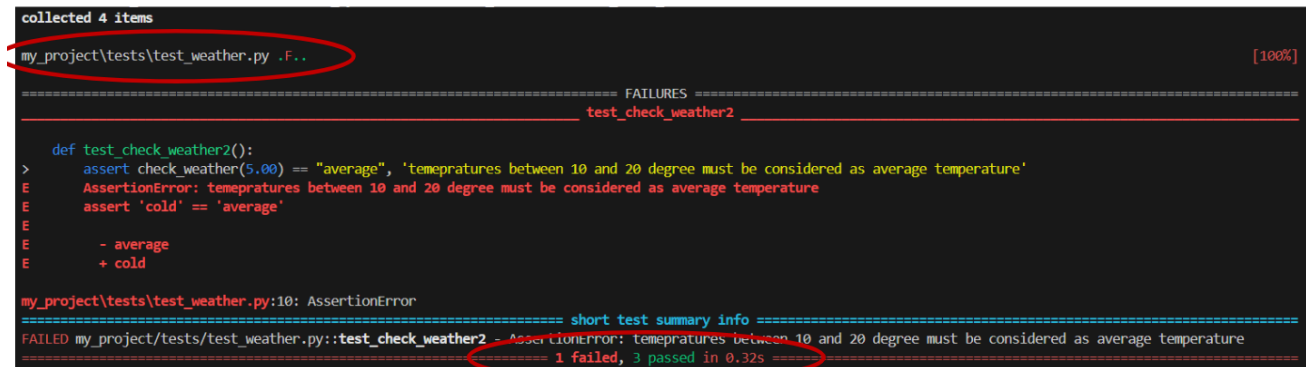
1  from my_project.weather import check_weather
2  #passed
3  def test_check_weather1():
4      assert check_weather(21.00) == "hot", "temperatures greater
   than 20 degree must be considered as hot"
5
6  #failed
7  def test_check_weather2():
8      assert check_weather(5.00) == "avarege", "temperatures
   between 10 and 20 degree must be considered as average
   temperature"
9
10 #passed
```

```

11 def test_check_weather3():
12     assert check_weather(5.00) == "cold", "temperatures lowe than
    10 degree must be considered as cold"
13
14 #passed
15 def test_check_weather1():
16     assert check_weather(13.00) == "average", "temperatures
    between 10 and 20 degree must be considered as average
    temperature"

```

L'output in questo caso sarà:



```

collected 4 items
my_project\tests\test_weather.py .F.. [100%]

===== FAILURES =====
test_check_weather2

def test_check_weather2():
    assert check_weather(5.00) == "average", 'temperatures between 10 and 20 degree must be considered as average temperature'
E   AssertionError: temperatures between 10 and 20 degree must be considered as average temperature
E   assert 'cold' == 'average'
E
E   - average
E   + cold
my_project\tests\test_weather.py:10: AssertionError

===== short test summary info =====
FAILED my_project\tests\test_weather.py::test_check_weather2 - AssertionError: temperatures between 10 and 20 degree must be considered as average temperature
===== 1 failed, 3 passed in 0.32s =====

```

Output pytest2.png

Ogni funzione `test_check_weather` definita è considerata da pytest come un singolo test che viene eseguito.

Infatti l'output mostra 4 test eseguiti: 1 fallito e 3 passati.

Tutti i nomi dei metodi devono iniziare con la parola `test`, questa è una convenzione che utilizziamo affinché **pytest** possa identificare i test che deve eseguire, se ad esempio scriviamo:



Python

```

1 from my_project.weather import check_weather
2 #passed
3 def not_test_check_weather1():
4     assert check_weather(21.00) == "hot", "temperatures greater
    than 20 degree must be considered as hot"
5
6 #failed
7 def test_check_weather2():
8     assert check_weather(5.00) == "avarege", "temperatures
    between 10 and 20 degree must be considered as average
    temperature"
9

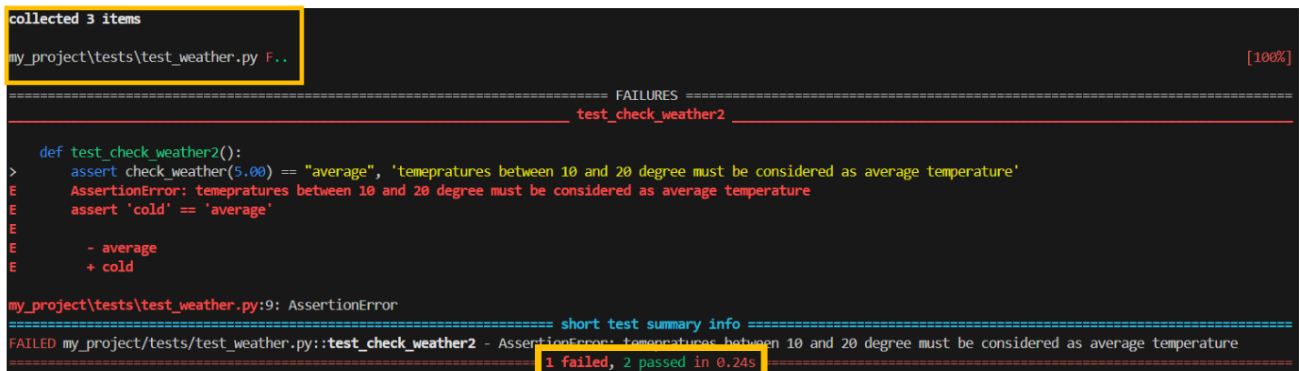
```

```

10 #passed
11 def test_check_weather3():
12     assert check_weather(5.00) == "cold", "temperatures lowe than
    10 degree must be considered as cold"
13
14 #passed
15 def test_check_weather4():
16     assert check_weather(13.00) == "average", "temperatures
    between 10 and 20 degree must be considered as average
    temperature"

```

L'output sarà:



```

collected 3 items
my_project\tests\test_weather.py F.. [100%]

===== FAILURES =====
test_check_weather2

def test_check_weather2():
> assert check_weather(5.00) == "average", 'temeptratures between 10 and 20 degree must be considered as average temperature'
E       AssertionError: temeptratures between 10 and 20 degree must be considered as average temperature
E       assert 'cold' == 'average'
E
E       - average
E       + cold
my_project\tests\test_weather.py:9: AssertionError

===== short test summary info =====
FAILED my_project\tests\test_weather.py::test_check_weather2 - AssertionError: temeptratures between 10 and 20 degree must be considered as average temperature
1 failed, 2 passed in 0.24s

```

Output pytest3.png

Py test trova solo 3 delle 4 funzioni che abbimao scritto, questo perché la prima funziona non usa il prefisso `test` nel nome quindi viene skippata.

Ora aggiungimao un altra funzione:



Python

```

1 from my_project.weather import check_weather
2 #passed
3 def test_check_weather1():
4     assert check_weather(21.00) == "hot", "temperatures greater
    than 20 degree must be considered as hot"
5
6 #failed
7 def test_check_weather2():
8     assert check_weather(5.00) == "avarege", "temperatures
    between 10 and 20 degree must be considered as average
    temperature"
9
10 #passed
11 def test_check_weather3():

```



```

12     assert check_weather(5.00) == "cold", "temperatures lower
    than 10 degree must be considered as cold"
13
14     #passed
15     def test_check_weather4():
16         assert check_weather(13.00) == "average", "temperatures
    between 10 and 20 degree must be considered as average
    temperature"
17
18     # failed because every def test_function() is considered as
    single test
19     def test_check_weather5():
20         assert check_weather(30.00) == "hot", "temperatures greater
    than 20 degree must be considered as hot"
21         assert check_weather(11.00) == "cold", "temperatures lower
    than 10 degree must be considered as cold"

```

L'output sar :

```

collected 5 items
my_project\tests\test_weather.py .F..F [100%]

===== FAILURES =====
test_check_weather2
def test_check_weather2():
> assert check_weather(5.00) == "average", 'temperatures between 10 and 20 degree must be considered as average temperature'
E       AssertionError: temeptratures between 10 and 20 degree must be considered as average temperature
E       assert 'cold' == 'average'
E
E       - average
E       + cold
my_project\tests\test_weather.py:9: AssertionError

test_check_weather5
def test_check_weather5():
    assert check_weather(30.00) == 'hot', 'temperatures greater than 20 degree must be considered as hot'
> assert check_weather(11.00) == 'cold', 'temperatures lower than 10 degree must be considered as cold'
E       AssertionError: temperatures lower than 10 degree must be considered as cold
E       assert 'average' == 'cold'
E
E       - cold
E       + average
my_project\tests\test_weather.py:22: AssertionError

===== short test summary info =====
FAILED my_project/tests/test_weather.py::test_check_weather2 - AssertionError: temeptratures between 10 and 20 degree must be considered as average temperature
FAILED my_project/tests/test_weather.py::test_check_weather5 - AssertionError: temperatures lower than 10 degree must be considered as cold
===== 2 failed, 3 passed in 0.26s =====

```

Output pytest4.png

L'output mostra 5 test eseguiti con 2 falliti e 3 passati.

Questo accade perch , anche se la funzione `test_check_weather5` contiene due istruzioni `assert`, essa viene comunque eseguita come un **singolo test**.

Pertanto, **ogni funzione** definita nel file di test viene eseguita come un **test unico**.

Pytest: test moltiplici

Quando si desidera testare una funzione con molte combinazioni di input e output, **senza scrivere molte funzioni di test separate**, il decoratore

`@pytest.mark.parametrize` risulta particolarmente utile.

`@pytest.mark.parametrize` è un **decoratore fornito da Pytest** che permette di eseguire **lo stesso test più volte con dati differenti**.

Perché utilizzarlo?

-  **Riduce la ripetizione del codice**
-  **Rende i test più leggibili e manutenibili**
-  **Facilita l'identificazione dei casi specifici che passano o falliscono**

Nota:

Per utilizzare `@pytest.mark.parametrize`, è necessario **importare il modulo** `pytest`.

Ma cosa fa `@pytest.mark.parametrize` ?

Esegue la funzione `test_check_weather` più volte, passando ogni volta una coppia di valori dalla lista fornita.

Questi valori sono:

- `temperature` : il valore di temperatura in ingresso (input)
- `expected` : il valore di output atteso (expected output)

QUindi nel file `test_weather.py` :



Python

```
1  from my_project.weather import check_weather
2  import pytest
3  @pytest.mark.parametrize("temperature, expected", [
4      (21.00, "hot"),
5      (13.00, "average"),
6      (0.00, "cold"),
7      (15.00, "cold")
8  ])
9  def test_check_weather(temperature, expected):
10     assert check_weather(temperature) == expected
```

`@pytest.mark.parametrize` esegue la funzione `test_check_weather` **4 volte**:

- La prima volta viene eseguita l'istruzione `assert check_weather(21.00) == "hot"` .
- La seconda volta viene eseguita l'istruzione `assert check_weather(13.00) == "average"` .
- La terza volta viene eseguita l'istruzione `assert check_weather(00.00) == "cold"` .
- La quarta volta viene eseguita l'istruzione `assert check_weather(15.00) == "cold"` .

Quindi l'output sarà:

```
collected 4 items
my_project\tests\test_weather.py ...F [100%]

===== FAILURES =====
test_check_weather[15.0-cold]
temperature = 15.0, expected = 'cold'

@pytest.mark.parametrize("temperature, expected", [
    (21.00, "hot"),
    (13.00, "average"),
    (0.00, "cold"),
    (15.00, "cold")
])
def test_check_weather(temperature, expected):
> assert check_weather(temperature) == expected
E   AssertionError: assert 'average' == 'cold'
E
E   - cold
E   + average
my_project\tests\test_weather.py:34: AssertionError

===== short test summary info =====
FAILED my_project/tests/test_weather.py::test_check_weather[15.0-cold] - AssertionError: assert 'average' == 'cold'
===== 1 failed, 3 passed in 0.26s =====
```

Output pytest5.png

Pytest esegue 4 test: 3 superano con successo e 1 fallisce, perché la quarta condizione verificata risulta falsa.

Grazie al decoratore `@pytest.mark.parametrize` , testare la funzione `check_weather()` diventa più **dinamico** e **meno ripetitivo**.

Inoltre si può anche assegnare un **messaggio personalizzato** di `AssertionError` se un test eseguito con `@pytest.mark.parametrize` fallisce. Questo aiuta a **comprendere meglio il contesto dell'errore** durante il debug.



Python

```
1 from my_project.weather import check_weather
2 import pytest
3 @pytest.mark.parametrize("temperature, expected", [
4     (21.00, "hot"),
5     (13.00, "average"),
6     (0.00, "cold"),
7     (15.00, "cold")
8 ])
```

```

9  def test_check_weather(temperature, expected):
10     ae:str = ""
11     if temperature >20:
12         ae = "temperatures greater than 20 degree must be
considered as hot"
13     elif 10 <temperature <=20:
14         ae = "temperature within 10 and 20 degree must be
considered as averege temperature"
15     else:
16         ae = "temperatures lower than 10 degree must be
considered as cold"
17     assert check_weather(temperature) == expected, ae

```

Andiamo a scrivere la classe `Calcolatore` in Python nel file `calculator.py`



Python

```

1  class Calculator:
2     def __init__(self, a:int, b:int):
3         self.a = a
4         self.b = b
5     def addition(self)->int:
6         return int(self.a + self.b)
7     def subtraction(self) ->int:
8         return int(self.a - self.b)
9     def multiplication (self)->int:
10        return int(self.a * self.b)
11    def division(self)->float|None:
12        if self.b == 0:
13            raise ValueError("Errore")
14        return round(self.a/self.b)

```

Ora nel file `test_calculator` scriviamo una serie test per la classe:



Python

```

1  from my_project.calculator import Calculator
2  def test_addition():
3      calculation:Calculator = Calculator (10,5)
4      assert calculation.addition() == 13, "The sum is wrong"
5
6  def test_subtraction():
7      calculation = Calculator(10,5)

```

```

8     assert calculation.multiplication() == 50, "The multiplication
    is wrong"
9
10    def test_division():
11        calculation = Calculator(10, 5)
12        assert calculation.division() == 2.00, "The quotient is wrong"
13

```

L'output sarà:

```

collected 4 items
my_project\tests\test_calculator.py F... [100%]

----- FAILURES -----
test_addition

def test_addition():
    calculation = Calculator(10, 5)
> assert calculation.addition() == 13, 'The sum is wrong'
E   AssertionError: The sum is wrong
E   assert 15 == 13
E   + where 15 = addition()
E   + where addition = <my_project.calculator.Calculator object at 0x0000015785D51250>.addition

my_project\tests\test_calculator.py:5: AssertionError
===== short test summary info =====
FAILED my_project\tests\test_calculator.py::test_addition - AssertionError: The sum is wrong
1 failed, 3 passed in 0.36s

```

Output pytest6.png

4 test eseguiti: 1 fallito e 3 passati

test_addiction è fallito.

Il decoratore @pytest.fixture

Probabilmente hai notato che, all'interno di ogni test, abbiamo inizializzato un oggetto della classe `Calculations`, che sarà testato.

Possiamo evitare questo passaggio utilizzando il decoratore `@pytest.fixture`.

Cos'è @pytest.fixture

`@pytest.fixture` è un **decoratore** che ti permette di creare **oggetti o risorse riutilizzabili** da utilizzare nei tuoi test.

Invece di creare un oggetto della classe `Calculator` (o simili) in ogni singolo test, questo oggetto viene creato **una sola volta** prima dell'esecuzione dei test, e poi **riutilizzato** in più test.

Serve a **preparare automaticamente** dati, oggetti, connessioni, file, ambienti, ecc. **prima che un test venga eseguito**, aiutandoti ad **evitare ripetizioni di codice**.

Quando usare @pytest.fixture ?

Uso quando:

- Hai bisogno di un oggetto (ad esempio un'istanza di classe, un dizionario, un file, ecc.) **in più test**
- Vuoi **separare** la configurazione/preparazione dalla logica del test
- Vuoi scrivere test **più chiari e facili da mantenere**

Esempio pratico di @pytest.fixture



Python

```
1  from my_project.calculator import Calculator
2  import pytest
3  @pytest.fixture
4  def calculation():
5      #Crea una nuova istanza della classe Calculator prima di
6      #ciascun test.
7      return Calculator(10,5)
8
9  def test_addition(calculation):
10     assert calculation.addition()==13, "The sum is wrong"
11
12 def test_subtraction(calculation):
13     assert calculation.subtraction()==5, "The subtraction is
14     wrong"
15
16 def test_multiplication(calculation):
17     assert calculation.multiplication()==50, "The multiplication
18     is wrong"
19
20 def test_division(calculation):
21     assert calculation.division()==2.00, "The quotient is wrong"
```

L'output sarà:

```
collected 4 items
my_project\tests\test_calculator.py F... [100%]

===== FAILURES =====
test_addition
def test_addition():
    calculation = Calculator(10,5)
    > assert calculation.addition() == 13, 'The sum is wrong'
E   AssertionError: The sum is wrong
E   assert 15 == 13
E   + where 15 = addition()
E   +   where addition = <my_project.calculator.Calculator object at 0x0000018785D51250>.addition

my_project\tests\test_calculator.py:5: AssertionError
===== short test summary info =====
FAILED my_project\tests\test_calculator.py::test_addition - AssertionError: The sum is wrong
===== 1 failed, 3 passed in 0.36s =====
```

Spiegazione:

4 test eseguiti: 1 fallito e 3 passati

`test_addiction` è fallito

Il comando Pytest

Per runnare tutti i test in una directory dobbiamo usare il comando `pytest` da terminale.

Questo comando permette:

- Cerca automaticamente tutti i file che **iniziano con** `test` oppure **terminano con** `_test.py`
- Esegue tutte le funzioni che **iniziano con** `test_`
- Supporta i test scritti con **pytest**

Il comando `pytest -v`

Il comando `pytest -v` viene utilizzato per eseguire i test in modalità "**verbosa**" con il framework **Pytest**.

Ovvero:

- Mostra l'elenco completo dei test eseguiti
- Visualizza anche i **parametri**, se si utilizza `@pytest.mark.parametrize`
- Per ogni test, mostra:
 - Il **nome del file**
 - Il **nome della funzione di test**
 - Il **risultato**: `FAILED` , `PASSED` , ecc.

Inoltre possiamo specificare un singolo test dentro una cartella e runnarlo usando il comando



Shell

```
1  pytest -v my_project/test/test_calculator.py
```

Oppure possiamo fare lo stesso con un singolo test di una funzione dentro un file di test:



Shell

```
1  pytest -v my_project/test/test/test_calculator.py::test_addition
```

L'output:



Shell

```
1  collected 1 item
2
3  my_project/test/test_calculator.py::test_addition FAILED
```

Nota

Nel comando `pytest`, i **due due punti** vengono utilizzati per **navigare all'interno di un file Python e specificare una particolare funzione di test da eseguire**.